

COMP3020 Assignment - Group 18

Annelise Tough, Neville Georgiades, Farhana Akter

2026-09-28

Group Member	Student ID	Contribution
Farhana Akter	22213545	
Neville Georgiades	22138296	
Annelise Tough	22091226	

Repository link: <https://github.com/farhanacg/COMP3020-Group18-Assignment>
(<https://github.com/farhanacg/COMP3020-Group18-Assignment>)

Introduction

This project explores how people discussed the FIFA World Cup 2026 on Bluesky, a social media platform. We used R and the `atrrr` package to collect 1,030 posts containing “FIFA World Cup” between 1 June and 31 July 2026.

However, most of the collected posts were from 30 July 2026 and focused on one news story about FIFA president Gianni Infantino’s plan to sell a private-equity stake in the World Cup and the possible boycott by UEFA and CONCACAF.

We used four methods to analyse the data: hypothesis testing, text analysis, clustering and network analysis. Our aim was to understand post engagement, common words and topics, groups of similar posts, and connections between users.

Analysis

Data Collection | Farhana

Choose a topic of interest and collect suitable data from an appropriate social web source using R. Document your data collection, including: • how the data was accessed and obtained; • relevant R packages; • variables collected and data volume; and • relevant limitations. Consider the limitations of your data source and collection process when interpreting your results.

Data source and collection

For this project, we collected social media data from **Bluesky** about the FIFA World Cup 2026. Bluesky was selected because it provides public social media posts that can be used to study people’s discussions and interactions around a topic.

We used the `atrrr` package in R to collect the data. The `atrrr` package provides functions for accessing Bluesky data through the AT Protocol. The main function used for this project was `search_skeet()`.

We searched for posts containing the phrase “**FIFA World Cup**” between **1 June 2026 and 31 July 2026**. The search was sorted by the latest posts, and we requested up to 1,000 posts.

The main code used was:

```
library(atrrr)
library("tm")
library("SnowballC")
library("wordcloud")

worldcup <- search_skeet(
  q = "FIFA World Cup",
  sort = "latest",
  since = "2026-06-01",
  until = "2026-07-31",
  limit = 1000L
)
```

After collecting the data, we checked the number of records using:

```
nrow(worldcup)
```

```
## [1] 1030
```

The result was 1,030 posts. Although the requested limit was 1,000, the function returned a small number of additional records. We kept all 1,030 collected posts for the analysis.

We also checked the variables available in the dataset:

```
names(worldcup)
```

```
## [1] "uri"          "cid"          "author_handle" "author_name"
## [5] "text"         "author_data"  "post_data"     "embed_data"
## [9] "reply_count"  "repost_count" "like_count"     "quote_count"
## [13] "bookmark_count" "indexed_at"   "created_at"     "in_reply_to"
## [17] "in_reply_root" "quotes"       "tags"           "mentions"
## [21] "links"        "langs"       "labels"
```

The data contained information about the posts, users, engagement, dates and relationships between posts.

Variables collected

Some of the main variables collected are shown below.

Variable	Description
uri	Unique identifier for each post
cid	Content identifier for the post
author_handle	Bluesky account handle of the user
author_name	Display name of the user
text	Text content of the post
reply_count	Number of replies to the post
repost_count	Number of reposts
like_count	Number of likes
quote_count	Number of quote posts
bookmark_count	Number of bookmarks
created_at	Date and time when the post was created
indexed_at	Date and time when the post was indexed
in_reply_to	Information about the post being replied to
in_reply_root	Root post of a reply conversation
quotes	Information about quoted posts
tags	Tags or hashtags associated with the post

Variable	Description
mentions	Accounts mentioned in the post
links	Links included in the post
langs	Language information
labels	Available content labels

The text variable is particularly important for this project because it will be used for text analysis and clustering. The engagement variables, such as `like_count`, `repost_count` and `reply_count`, can also be used to examine how users interacted with the posts.

Data volume

The final dataset contained 1,030 Bluesky posts collected during the selected period from 1 June 2026 to 31 July 2026.

The dataset contains several list-type variables, including `author_data`, `post_data`, `embed_data`, `tags`, `mentions`, `links`, `langs` and `labels`. These variables contain structured information about posts and users. The original dataset was kept in R because some of this information may be useful for other parts of the project, particularly network analysis.

504 timeout issue

During the initial data collection, the request returned a **504 Gateway Timeout** error when the same search was attempted. This means that the request did not receive a response from the server within the expected time.

The search was tried again later using the same `search_skeet()` method, and it worked successfully. Therefore, the 504 error was treated as a temporary connection or server response issue rather than a problem with the R code.

Limitations

There are several limitations to this data collection. First, the data only comes from Bluesky, so it does not represent discussions from other social media platforms or the opinions of the general population.

Second, we used the search phrase “FIFA World Cup”. Posts discussing the World Cup using different words or terms may not have been included. Search results can also match information connected to a post, such as a link preview or image alt text, and not only the main post text.

Another limitation is that engagement numbers can change over time. For example, a post may receive more likes, reposts or replies after the data was collected. Therefore, the engagement values represent the information available when the data was collected.

Finally, the dataset is a sample of Bluesky posts rather than every post related to the FIFA World Cup. Therefore, the results should be understood as patterns within our collected dataset rather than a complete representation of public opinion about the FIFA World Cup.

Saving

As the project is being completed on multiple devices, it is important to make sure the same data is being used. We used the below code to save the data, and then this file was shared with the other group members.

```
#save(worldcup, compress = FALSE, file = "/Users/annelisetough/Desktop/WSU/COMP3020/WorldCupSkeets.RData")
# This has been commented out as the data has already been saved
load("WorldCupSkeets.RData")
```

Data Cleaning | Neville

```
library(tidyverse)
library(tidytext)
library(wordcloud)
library(RColorBrewer)
```

Checking the data before cleaning

```
nrow(worldcup) #posts before cleaning
```

```
## [1] 1030
```

```
sum(is.na(worldcup$text)) #post with missing text
```

```
## [1] 0
```

```
sum(duplicated(worldcup$uri)) #duplicate posts
```

```
## [1] 0
```

Cleaning

```
wc <- worldcup%>%
distinct(uri, .keep_all = TRUE) %>% #remove duplicates
filter(!is.na(text), str_trim(text) != "") #remove missing/empty text
wc <- wc %>%
mutate(
date = as.Date(created_at),
post_length = nchar(text),
clean_text = text %>%
str_to_lower() %>% # lowercase
str_remove_all("https?://\\S+") %>% # remove links
str_remove_all("@\\S+") %>% # remove @mentions
str_replace_all("[^a-z\\s]", " ") %>% # keep letters only
str_squish(), # remove extra spaces
word_count = str_count(clean_text, "\\S+"),
# total engagement = likes + reposts + replies + quotes
total_engagement = like_count + repost_count + reply_count + quote_count
) %>%
filter(clean_text != "") # remove posts empty after cleaning
nrow(wc) # posts after cleaning
```

```
## [1] 998
```

English posts only (the keyword search and stop words only work in English)

```
wc_en <- wc %>%
  filter(sapply(langs, function(x) "en" %in% x))
nrow(wc_en) # English posts
```

```
## [1] 793
```

Hypothesis Testing | Neville

Develop a research question that can be addressed using hypothesis testing. The research question may involve any suitable variables in the data collected from the social web, but should contribute to the overall investigation. Clearly state the hypotheses and select an appropriate statistical test from those covered in COMP3020. Explain why the test is suitable and interpret the result in the context of your research question. Hint: Consider whether characteristics of posts, users, films or other entities in your dataset can be compared or associated in a way that addresses an interesting question.

Research question Do posts mentioning a host country (USA, Canada, Mexico) differ in engagement from posts that don't?

```
wc_en <- wc_en %>%
  mutate(
    mentions_host = str_detect(
      clean_text,
      "\\b(usa|united states|canada|mexico)\\b"
    ),
    host_group = if_else(mentions_host, "Mentions host country", "No host country"),
    host_group = factor(host_group,
      levels = c("No host country", "Mentions host country"))
  )
table(wc_en$host_group)
```

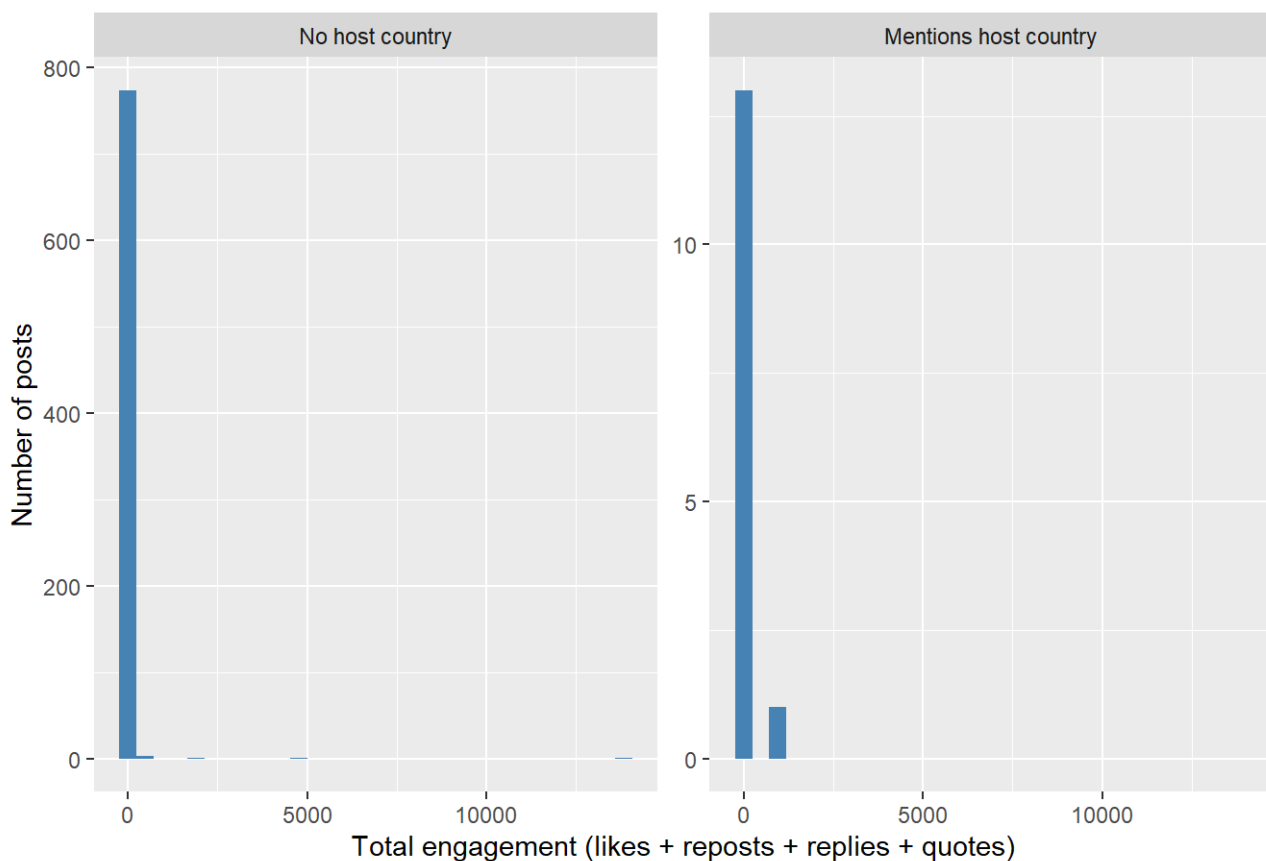
```
##
##          No host country Mentions host country
##                779                14
```

```
wc_en %>%
  group_by(host_group) %>%
  summarise(
    n = n(),
    mean_engagement = mean(total_engagement),
    median_engagement = median(total_engagement),
    max_engagement = max(total_engagement),
    pct_any_engagement = round(mean(total_engagement > 0) * 100, 1)
  )
```

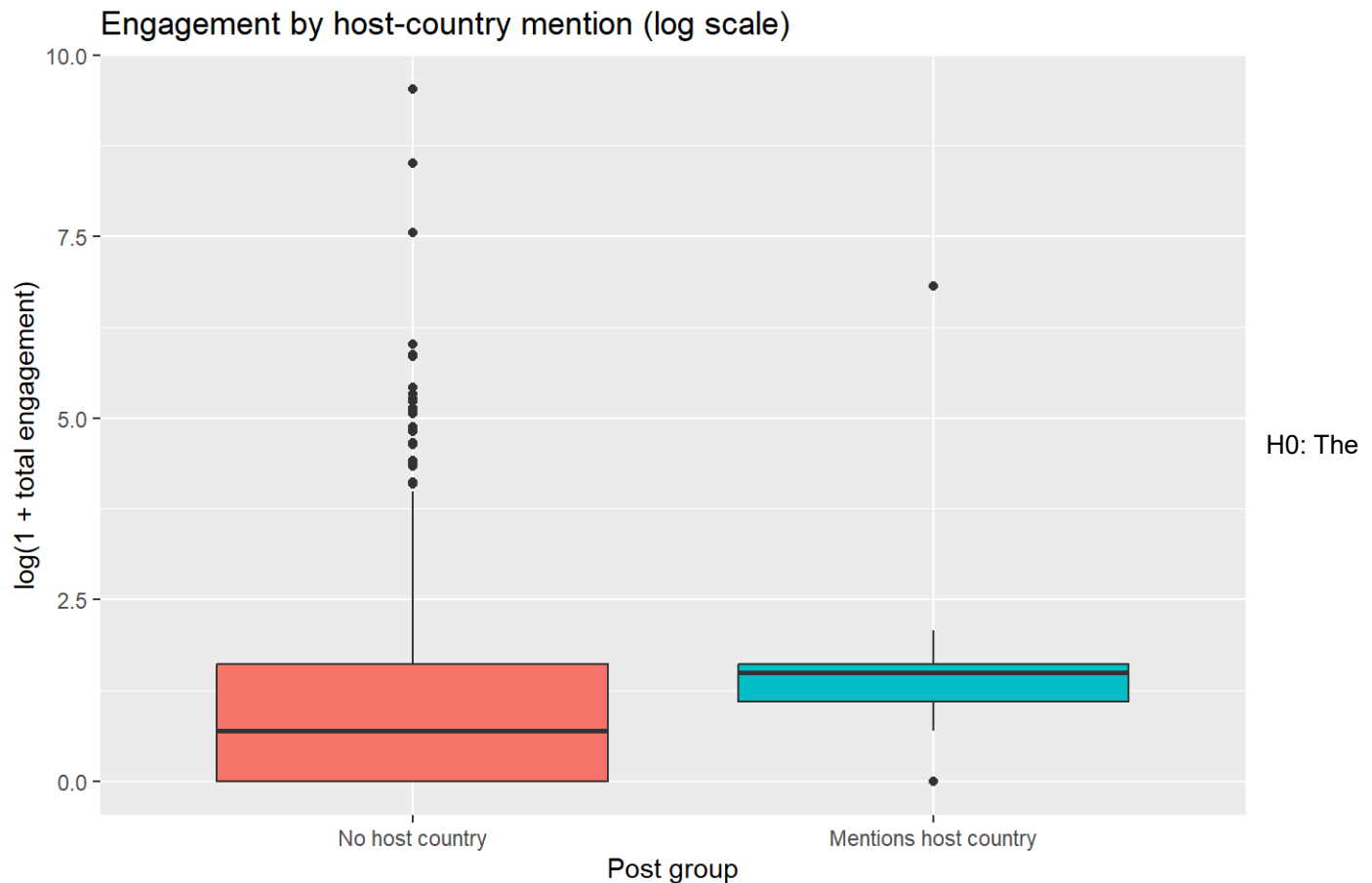
```
## # A tibble: 2 × 6
##   host_group          n mean_engagement median_engagement max_engagement
##   <fct>          <int>          <dbl>          <dbl>          <int>
## 1 No host country    779            34.5            1          13823
## 2 Mentions host country  14            67.9            3.5           911
## # 1 more variable: pct_any_engagement <dbl>
```

```
ggplot(wc_en, aes(x = total_engagement)) +
  geom_histogram(bins = 30, fill = "steelblue") +
  facet_wrap(~ host_group, scales = "free_y") +
  labs(title = "Distribution of engagement by host-country mention",
       x = "Total engagement (likes + reposts + replies + quotes)",
       y = "Number of posts")
```

Distribution of engagement by host-country mention



```
ggplot(wc_en, aes(x = host_group, y = log1p(total_engagement), fill = host_group)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Engagement by host-country mention (log scale)",
       x = "Post group", y = "log(1 + total engagement)")
```



distribution of engagement is the same for posts that mention a host country and posts that do not. H1: The distribution of engagement differs between the two groups. Test: Wilcoxon rank-sum (Mann-Whitney U) test. The two groups are independent, and the plots above show engagement is heavily right-skewed with many zeros, so the normality assumption of a t-test is not met.

```
wilcox.test(total_engagement ~ host_group, data = wc_en, exact = FALSE)
```

```
##
## Wilcoxon rank sum test with continuity correction
##
## data: total_engagement by host_group
## W = 3792.5, p-value = 0.04406
## alternative hypothesis: true location shift is not equal to 0
```

Interpretation: We used a Wilcoxon rank-sum test to check whether posts mentioning a host country (USA, Canada or Mexico) had different engagement levels. The test returned $W = 3792.5$ and $p = 0.044$. Since the p-value is below 0.05, we rejected the null hypothesis and found a statistically significant difference between the two groups.

Posts mentioning a host country had higher engagement, with a mean of 67.9 and median of 3.5 interactions. In comparison, other posts had a mean of 34.5 and median of 1.0. Also, 85.7% of host-country posts received engagement, compared to 62.8% of other posts.

However, only 14 posts mentioned a host country, compared to 779 others. A few popular posts may have influenced the results, and the p-value was close to 0.05. We also may have missed posts referring to countries indirectly. Therefore, these findings only apply to our sample of English-language Bluesky posts and do not prove that mentioning a host country causes higher engagement.

Text/Content Analysis and Visualisation | Neville

Clean the data and address missing or unsuitable records. Explore the textual/content characteristics of your data and represent the text in an appropriate form for further analysis. Provide relevant summaries and at least two informative visualisations, including a visualisation of frequent words. Interpret the patterns you identify and explain how they contribute to your investigation.

Text summaries

```
summary(wc_en$post_length)
```

##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
##	12.0	74.0	151.0	159.3	249.0	319.0

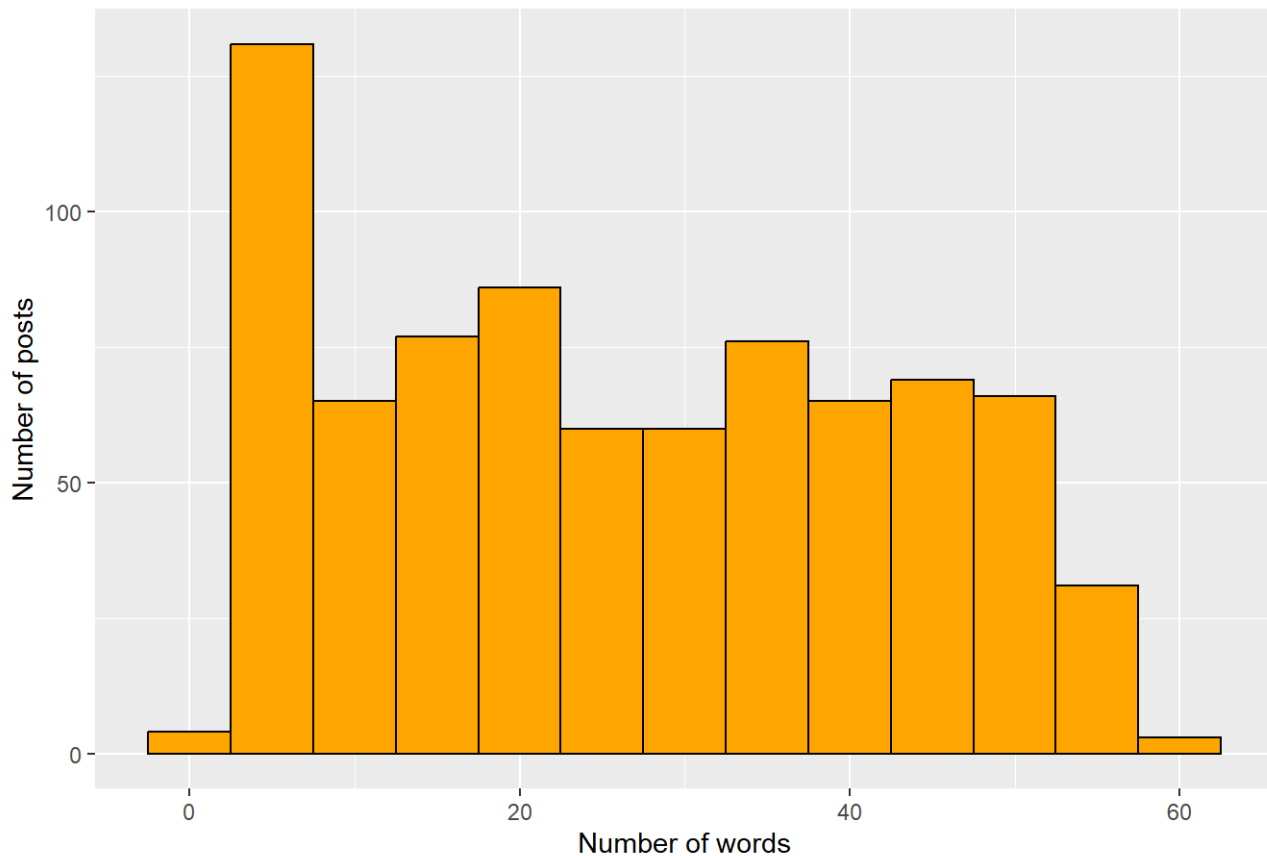
```
summary(wc_en$word_count)
```

##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
##	1.0	12.0	25.0	26.5	40.0	59.0

histogram of the number of words in each post

```
ggplot(wc_en, aes(x = word_count)) +  
  geom_histogram(binwidth = 5, fill = "orange", colour = "black") +  
  labs(title = "Number of Words per Post",  
        x = "Number of words",  
        y = "Number of posts")
```

Number of Words per Post



Interpretation

The 793 English posts varied in length, ranging from 12 to 319 characters. The median word count was 25, with an average of 26.5 words per post. Since the mean and median are close, the word counts were fairly balanced overall.

The histogram showed that many posts contained 5 words or fewer, while most others were between 10 and 50 words. Posts longer than 50 words were less common. The shorter posts may mainly contain links, hashtags or mentions rather than longer text.

Word frequencies

#words to remove: our search terms and some filler words

```
custom_stop <- tibble(word = c("fifa", "world", "cup", "worldcup",
"fifaworldcup", "amp", "just", "will",
"like", "get", "got", "www", "footba"))
#split each post into single words and remove stop words
```

```
tokens <- wc_en %>%
select(uri, clean_text) %>%
unnest_tokens(word, clean_text) %>%
anti_join(stop_words, by = "word") %>%
anti_join(custom_stop, by = "word") %>%
filter(nchar(word) > 2)
#count how many times each word appears
```

```
word_freq <- tokens %>%
count(word, sort = TRUE)
head(word_freq, 20) # top 20 words
```

A tibble: 20 × 2

##	word	n
##	<chr>	<int>
##	1 uefa	342
##	2 infantino	205
##	3 boycott	195
##	4 football	149
##	5 sell	125
##	6 trump	122
##	7 sport	120
##	8 plan	113
##	9 private	107
##	10 bbc	95
##	11 european	83
##	12 competitions	73
##	13 kushner	72
##	14 europe	71
##	15 sports	70
##	16 news	68
##	17 investors	67
##	18 concacaf	66
##	19 nations	62
##	20 soccer	60

```
nrow(word_freq) # number of different words
```

```
## [1] 2627
```

Interpretation

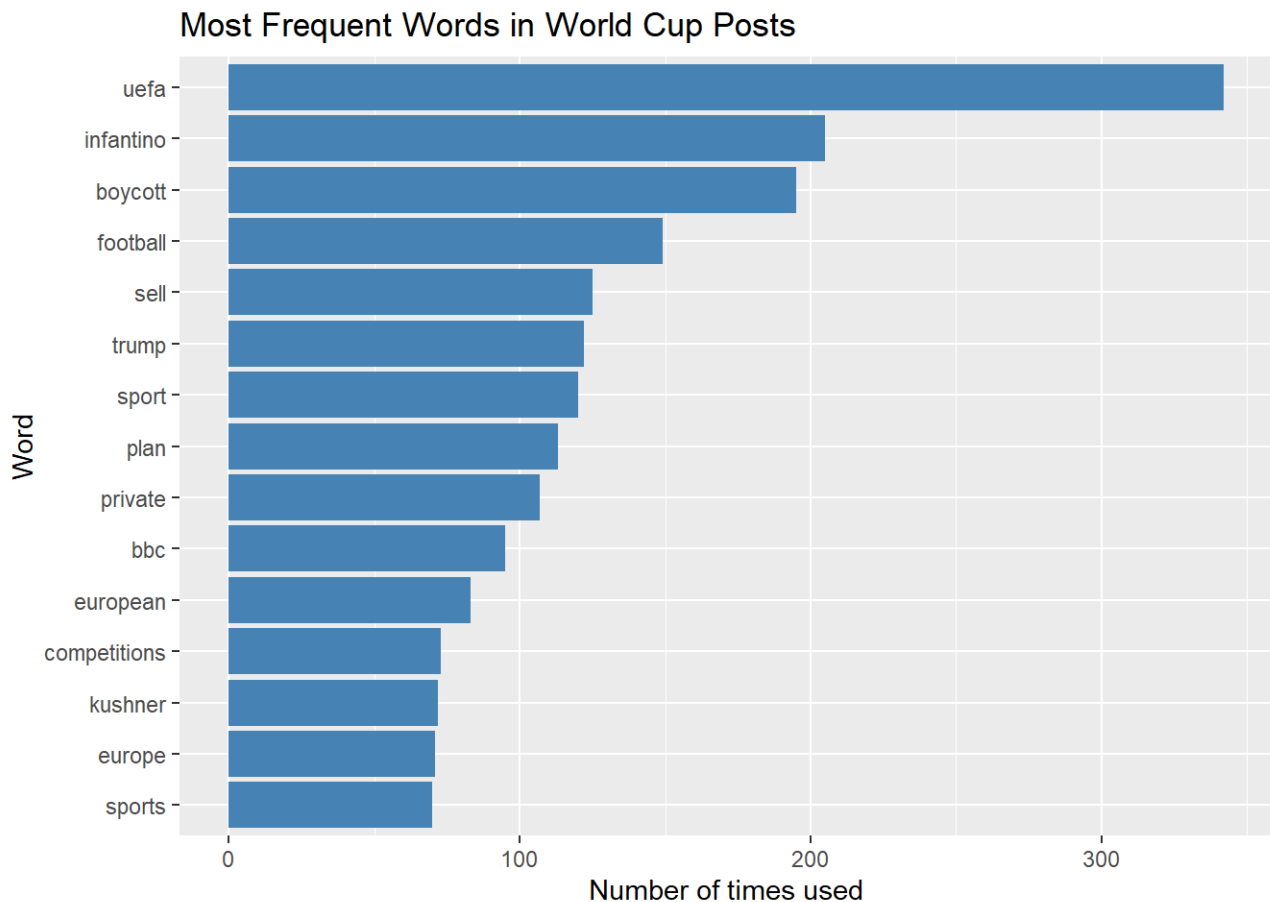
The most common words in the English posts were “uefa” (342 times), “infantino” (205), “boycott” (195) and “football” (149). Other frequent words included “trump”, “private”, “investors” and “kushner”. This suggests that many posts focused on a specific news story involving UEFA, FIFA president Gianni Infantino, private investors and a possible boycott, rather than general World Cup discussion.

This means the dataset may not represent World Cup discussions overall, as many posts appear to focus on the same topic. This could also affect the clustering results, as posts may be grouped based on this news story. Words such as “footba” and “www” were removed because they came from cut-off text or web addresses.

Visualisations

#bar chart of the 15 most common words

```
word_freq %>%
  slice_max(n, n = 15) %>%
  ggplot(aes(x = reorder(word, n), y = n)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Most Frequent Words in World Cup Posts",
       x = "Word",
       y = "Number of times used")
```

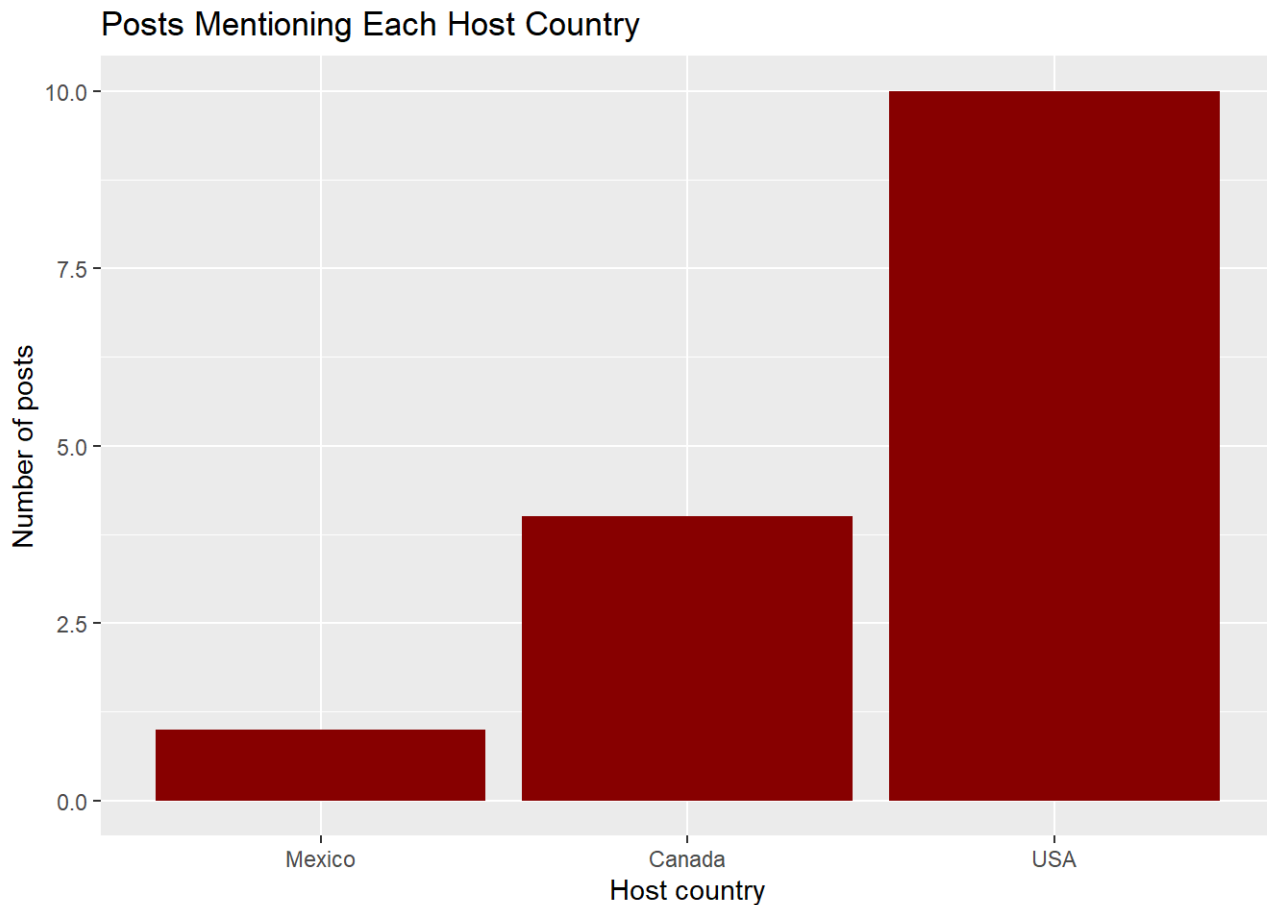


```
set.seed(123)
wordcloud(words = word_freq$word, freq = word_freq$n,
max.words = 80, min.freq = 3,
random.order = FALSE,
colors = brewer.pal(8, "Dark2"))
```



```
host_counts <- tibble(
  country = c("USA", "Canada", "Mexico"),
  posts = c(
    sum(str_detect(wc_en$clean_text, "\\b(usa|united states)\\b")),
    sum(str_detect(wc_en$clean_text, "\\bcanda\\b")),
    sum(str_detect(wc_en$clean_text, "\\bmexico\\b"))
  )
)

ggplot(host_counts, aes(x = reorder(country, posts), y = posts)) +
  geom_col(fill = "darkred") +
  labs(title = "Posts Mentioning Each Host Country",
       x = "Host country",
       y = "Number of posts")
```



Interpretation

The bar chart and word cloud show that the most common words were “uefa”, “infantino”, “boycott” and “football”. This suggests that most posts focused on a news story involving UEFA, FIFA president Gianni Infantino and a possible boycott, rather than matches or players.

The date table shows that almost all posts were from 30 July 2026. This means the data only represents discussion from one day, possibly because the search returned the latest posts first and stopped at around 1,000 posts.

The USA was mentioned in 10 posts, Canada in 4 and Mexico in 1. These total more than 14 because some posts mentioned multiple countries. Overall, the data focuses heavily on one news story, which limits the hypothesis test and may affect the clustering results.

Clustering

Investigate whether meaningful clusters can be identified in the textual/content data. Make appropriate decisions about the representation of the data, distance measure and number of clusters, and justify these decisions. Visualise your clusters and summarise the themes or characteristics of each cluster. Your clustering analysis should go beyond simply running an R function and reporting the cluster labels.

Given that the previous results indicate the main topic of the skeets revolve around a particular news story, we can assess if clustering also suggests this.

First we must build a corpus from the skeets that we can eventually transform into a matrix.

```

worldcup.corpus=Corpus(VectorSource(wc_en$clean_text))
worldcup.corpus = tm_map(
  worldcup.corpus,
  function(x) iconv(x, to = "ASCII")
)
# We will remove more words before continuing
custom_words = c("com", "www", "org", "fuck", "shit")
worldcup.corpus = tm_map(worldcup.corpus, removeWords, c(stopwords("english"), custom_words, tolower(custom_words)))
# Extract document vectors and apply TF-IDF weighting
## Create a term-document matrix
worldcup.tdm=TermDocumentMatrix(worldcup.corpus)
# Apply TF-IDF weighting.
worldcup.wtdm = weightTfIdf(worldcup.tdm)
# Transpose so documents are rows and terms are columns.
worldcup.matrix = t(as.matrix(worldcup.wtdm))
# Remove empty texts
empties = which(rowSums(abs(worldcup.matrix)) == 0)
empties

```

```
## named integer(0)
```

```
# There are no empty rows, so we don't need to apply this to the matrix
```

K-Means Clustering

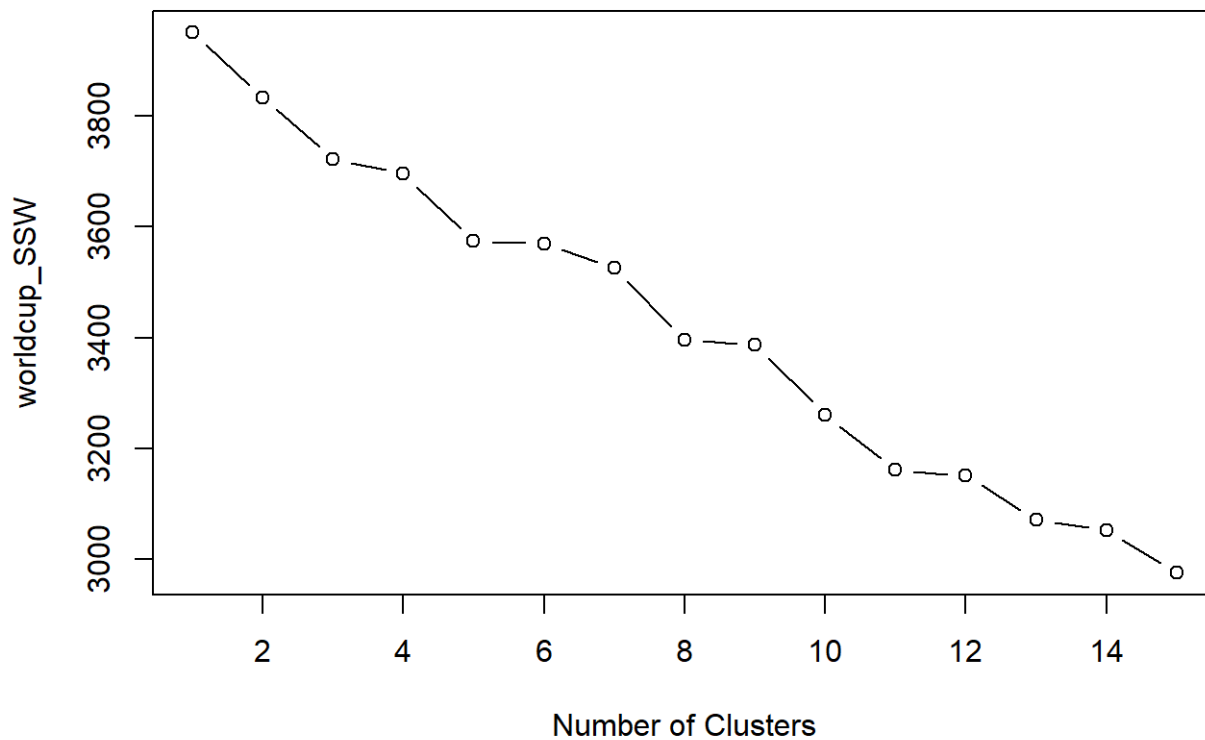
When using K-means clustering, we need to decide the appropriate amount of clusters. We can observe the within sum of squares (SSW) and choose the number of clusters based on which K value has the lowest SSW, or where we see an “elbow” in the plot of the SSWs.

We start by observing the SSWs for clusters of size 1 to 15.

```

worldcup_SSW <- list()
for(i in 1:15){
  K = kmeans(worldcup.matrix, i)
  worldcup_SSW[i] <- K$tot.withinss
}
plot(1:15, worldcup_SSW, type = "b", xlab = "Number of Clusters")

```



From this plot, we see an elbow at 11 clusters.

This K-means test used Euclidean distance. Let's see if there is a difference when using cosine distance.

```
# K-Means clustering with cosine distance
norm.worldcup.matrix=diag(
  1/sqrt(rowSums(worldcup.matrix^2))
) %%% worldcup.matrix

# create manual cosine distance
D = dist(norm.worldcup.matrix, method="euclidean")^2/2

N = dim(worldcup.matrix)[1]
N
```

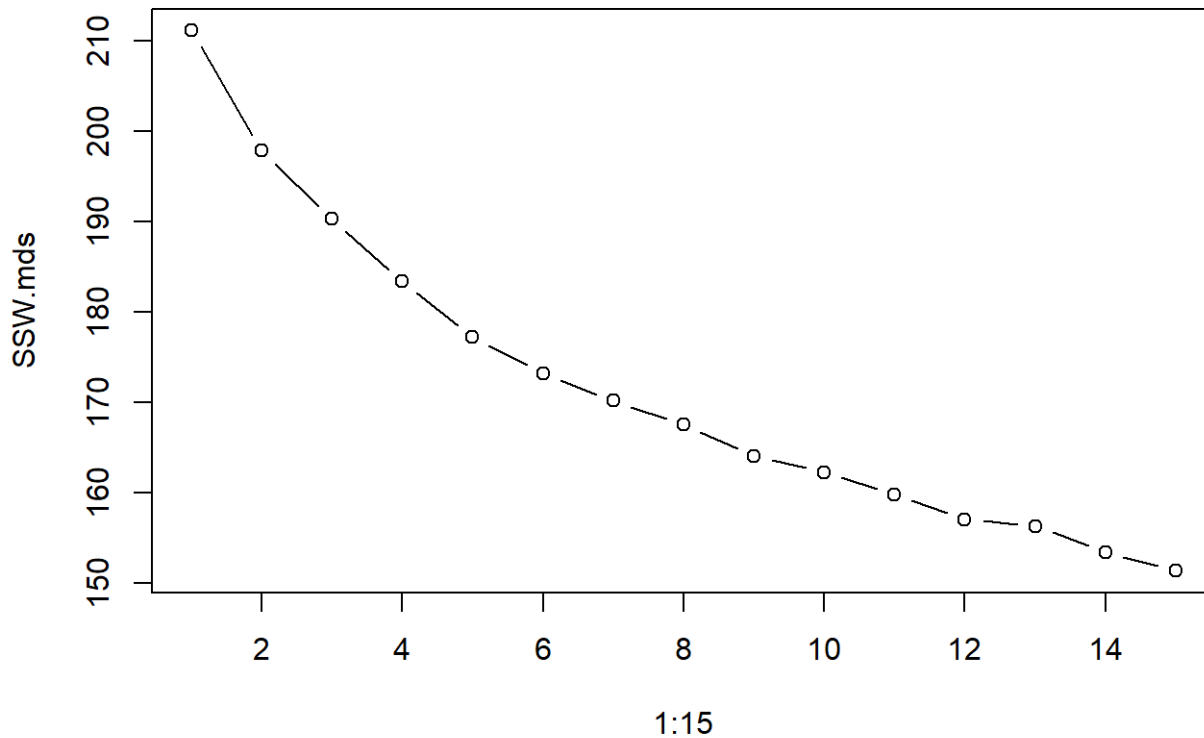
```
## [1] 793
```

```
dim(as.matrix(D))
```

```
## [1] 793 793
```

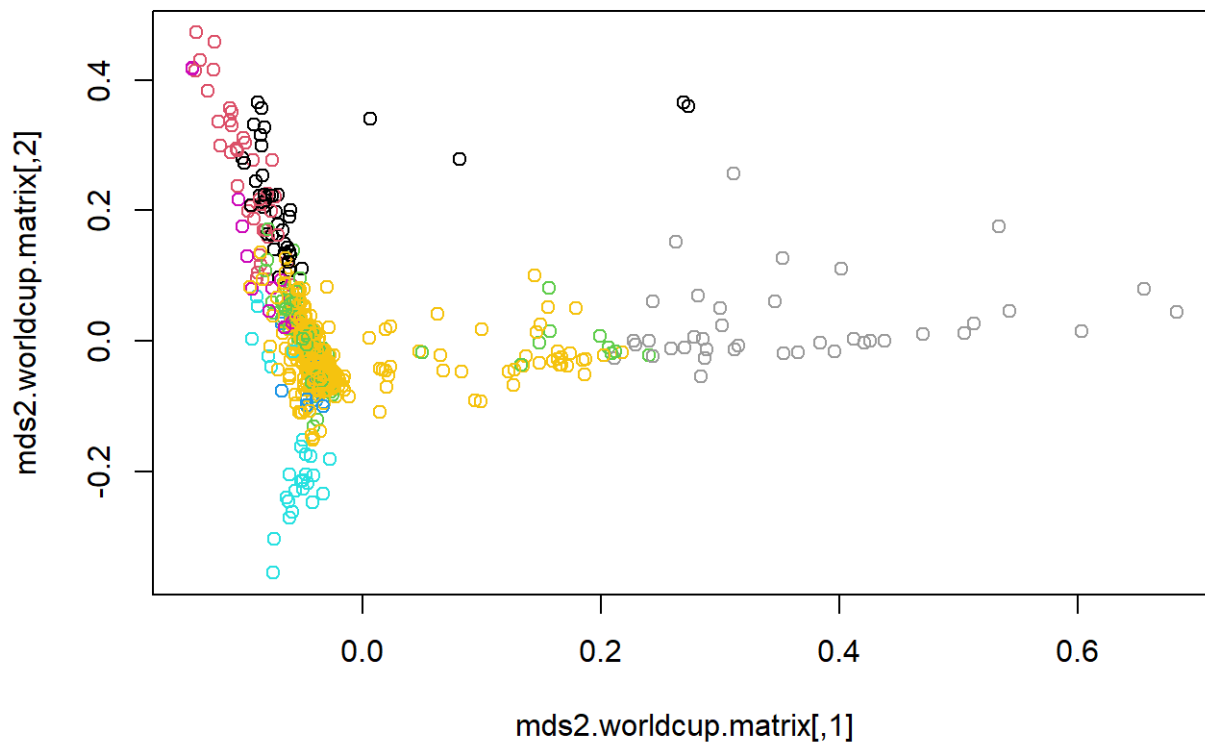
```
mds.worldcup.matrix=cmdscale(D, k=87)

SSW.mds = rep(0,15)
for(i in 1:15){
  K=kmeans(mds.worldcup.matrix, i, nstart = 20)
  SSW.mds[i] = K$tot.withinss
}
plot(1:15, SSW.mds, type = "b")
```



There isn't an obvious elbow in this plot, but we could choose a cluster from about 8 upwards. We will choose 8 for now.

```
K=kmeans(mds.worldcup.matrix, 8, nstart=20)
mds2.worldcup.matrix=cmdscale(D,k=2)
plot(mds2.worldcup.matrix, col = K$cluster)
```



It is evident from this plot that most skeets contain similar words/content as the points are largely bunched.

```
# Examining the cluster contents
table(K$cluster)
```

```
##
##  1  2  3  4  5  6  7  8
## 41 44 84 22 35 23 487 57
```

```

explore_clusters <- function(x){
  # Find top terms for the first cluster
  cluster.number = x
  #find posts assigned to this cluster
  clusterpostsId=which(K$cluster == cluster.number)
  #extract their original weighted document vectors
  clusterposts = worldcup.matrix[clusterpostsId,]
  #calculate the mean TF-IDF weight of each term
  clusterTermWeight=colMeans(clusterposts)
  #display the top 10 weighted terms
  print(sort(clusterTermWeight, decreasing = TRUE)[1:10])
  #make wordcloud
  wordcloud(
    words = names(clusterTermWeight),
    freq = clusterTermWeight,
    min.freq = 3,
    max.words = 100,
    random.order = FALSE,
    rot.per = 0.35,
    colors = brewer.pal(8, "Dark2"),
    scale = c(2, .5)
  )
}
explore_clusters(1)

```

##	investors	private	stakes	competitions	sell	associations
##	0.13419045	0.11730781	0.11565583	0.09882765	0.09878167	0.09862315
##	boycott	member	voted	uefa		
##	0.09572295	0.08808235	0.08279231	0.08226988		



Construct a network from meaningful relationships in the data, such as follows, replies, mentions, reposts, film–actor links, film–director links or actor co-appearances. The relationships represented by the edges must come from the data; arbitrary relationships created simply to satisfy the network requirement are not acceptable. Aim to construct a sufficiently large and well-connected network that allows meaningful analysis.

- Define nodes and edges, explain what edges represent and justify whether the graph is directed or undirected.
- Report node and edge counts and relevant properties such as density, components or degree distribution.
- Visualise the full network and a justified subgraph of the network. Use layout, colour, labels and size effectively.
- Calculate and interpret at least two centrality measures; identify and discuss important nodes.
- Explain how data collection, API limitations, and any filtering or network construction decisions may affect your findings.

We chose to construct a Bluesky network based on skeet likes. Our seed user is the user whose post has the highest like count out of all the skeets in our database. From there, we will identify a portion of the people who liked that skeet, then extract some of their own skeets and find people who have liked those.

20/32

```
library(dplyr)
# Create new worldcup data frame that is ordered from most likes to least likes
worldcup_likes_order = worldcup[order(worldcup$like_count, decreasing = TRUE),]

# Create author info data frame where users are in order of likes
author_info_likes <- get_user_info(unique(worldcup_likes_order$author_handle))

# Display the five authors with the most likes
top_candidates <- head(author_info_likes[, c("actor_handle", "actor_name", "follows_count")], 5)
top_candidates
```

```
## # A tibble: 5 × 3
##   actor_handle      actor_name    follows_count
##   <chr>           <chr>          <int>
## 1 itismashagain.bsky.social Mash             1876
## 2 cascoinfoundation.org H.E. Cas Piancey    533
## 3 johnswinney.bsky.social John Swinney         76
## 4 aaronparnas.bsky.social Aaron Parnas        293
## 5 npr.org          NPR             238
```

From the top 5 candidates, we can see that a normal user, `isismashagain.bsky.social` has the most likes and that the most liked news skeet comes from `npr.org`.

```
# Our normal seed user will be the first row of our author_info_likes data frame
seed_normal_likes_handle <- author_info_likes$actor_handle[1]
seed_news_likes_handle <- author_info_likes$actor_handle[5]

# Find the seed users posts
seed_normal_likes_post <- worldcup_likes_order[1,]
seed_news_likes_post <- worldcup_likes_order[5,]

# Now we can see who liked this post. We limit the search to 20 users for the sake of interpretation.
normal_likes_handles <- get_likes(seed_normal_likes_post$uri, limit = 20)$actor_handle
head(normal_likes_handles)
```

```
## [1] "midnightambassador.bsky.social" "patrickandrews.bsky.social"
## [3] "thisis-insane.bsky.social"     "maiab-123.bsky.social"
## [5] "thestampedehero.bsky.social"   "a-duck-enjoyer.bsky.social"
```

```
news_likes_handles <- get_likes(seed_news_likes_post$uri, limit = 20)$actor_handle
head(news_likes_handles)
```

```
## [1] "vetothepecocheeto.bsky.social" "firstmillenial.bsky.social"
## [3] "martinrca.bsky.social"         "durgs1.bsky.social"
## [5] "uglydook.bsky.social"          "pfalcon.bsky.social"
```

```
# Collect a selection of skeets from each of the normal_likes_handles feeds
normal_feeds <- list()
for(i in 1:length(normal_likes_handles)){
  normal_feeds[[i]] <- data.frame(get_skeets_authored_by(normal_likes_handles[i], limit = 2))
} # we limit to 2 as otherwise there is too much data to handle
# Transform the list into a dataframe
normal_likes_feeds_df <- do.call(rbind, normal_feeds)
## Note that not all of the skeets a user has posted are originally theirs, some are reposts.
dim(normal_likes_feeds_df)[1]
```

```
## [1] 78
```

```
length(normal_likes_feeds_df)
```

```
## [1] 24
```

```
# Find who has liked posts from the Likes_handles.
normal_more_likes <- list()
for(i in 1:dim(normal_likes_feeds_df)[1]){
  normal_more_likes[[i]] <- get_likes(normal_likes_feeds_df[i,]$uri, limit = 10)
}
normal_more_likes_handles <- lapply(normal_more_likes, '[', "actor_handle")
sapply(normal_more_likes_handles, length)
```

```
## [1] 0 1 3 10 3 0 0 0 6 0 10 10 10 1 10 10 10 10 2 0 1 2 3 2 0
## [26] 0 10 10 1 3 10 10 10 1 8 6 3 0 10 10 3 10 0 0 10 10 0 1 2 19
## [51] 2 5 10 0 0 10 1 5 1 2 1 0 1 1 10 10 4 4 10 10 10 2 10 0 7
## [76] 10 19 17
```

```
# Repeat process for news_likes_handles
news_feeds <- list()
for(i in 1:length(news_likes_handles)){
  news_feeds[[i]] <- data.frame(get_skeets_authored_by(news_likes_handles[i], limit = 2))
}
news_likes_feeds_df <- do.call(rbind, news_feeds)
dim(news_likes_feeds_df)[1]
```

```
## [1] 38
```

```
length(news_likes_feeds_df)
```

```
## [1] 24
```

```
news_more_likes <- list()
for(i in 1:dim(news_likes_feeds_df)[1]){
  news_more_likes[[i]] <- get_likes(news_likes_feeds_df[i,]$uri, limit = 10)
}
news_more_likes_handles <- lapply(news_more_likes, '[', "actor_handle")
sapply(news_more_likes_handles, length)
```

```
## [1] 1 0 0 1 10 10 0 0 3 0 6 0 8 7 10 0 0 0 7 19 10 6 0 1 18
## [26] 10 0 18 1 2 10 0 10 10 10 10 1 10
```

```
# Create directed edge list for normal users
## Make one row for each edge from the seed user to a liker
normal_edgelist_seed <- cbind(
  from=rep(seed_normal_likes_handle, length(normal_likes_handles)),
  to=normal_likes_handles
)

## Make the second-level edges, one liker at a time
normal_edgelist_friends <- lapply(seq_along(normal_likes_handles), function(i) {
  cbind(from =rep(normal_likes_handles[i],
    length(normal_more_likes_handles[[i]])),
    to=normal_more_likes_handles[[i]])
})
# Join the first and second level edges into one edge list
normal_edgelist <- unique(rbind(normal_edgelist_seed, do.call(rbind,normal_edgelist_friends)))
# Remove self-likes, where an account would point to itself
normal_edgelist <- normal_edgelist[normal_edgelist[, "from"] != normal_edgelist[, "to"], , drop = FALSE]

# Show the number of rows and columns in the final edge list
dim(normal_edgelist)
```

```
## [1] 225 2
```

```
# Preview the first few follow relationships
head(normal_edgelist)
```

```
##      from                                to
## [1,] "itismashagain.bsky.social" "midnightambassador.bsky.social"
## [2,] "itismashagain.bsky.social" "patrickandrews.bsky.social"
## [3,] "itismashagain.bsky.social" "thisis-insane.bsky.social"
## [4,] "itismashagain.bsky.social" "maiab-123.bsky.social"
## [5,] "itismashagain.bsky.social" "thestampedehero.bsky.social"
## [6,] "itismashagain.bsky.social" "a-duck-enjoyer.bsky.social"
```

```
# Repeat edgelists for news users
news_edgelist_seed <- cbind(
  from=rep(seed_news_likes_handle, length(news_likes_handles)),
  to=news_likes_handles
)
## Make the second-level edges, one liker at a time
news_edgelist_friends <- lapply(seq_along(news_likes_handles), function(i) {
  cbind(from =rep(news_likes_handles[i],
    length(news_more_likes_handles[[i]])),
    to=news_more_likes_handles[[i]])
})
# Join the first and second level edges into one edge list
news_edgelist <- unique(rbind(news_edgelist_seed, do.call(rbind,news_edgelist_friends)))
# Remove self-likes, where an account would point to itself
news_edgelist <- news_edgelist[news_edgelist[, "from"] != news_edgelist[, "to"], , drop = FALSE]

# Show the number of rows and columns in the final edge list
dim(news_edgelist)
```

```
## [1] 102  2
```

```
# Preview the first few follow relationships
head(news_edgelist)
```

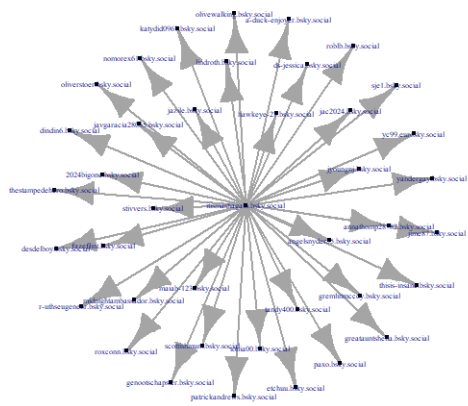
```
##      from      to
## [1,] "npr.org" "vetothepedocheeto.bsky.social"
## [2,] "npr.org" "firstmillenial.bsky.social"
## [3,] "npr.org" "martinrca.bsky.social"
## [4,] "npr.org" "durgs1.bsky.social"
## [5,] "npr.org" "uglydook.bsky.social"
## [6,] "npr.org" "pfalcon.bsky.social"
```

```
#Build and plot the network
```

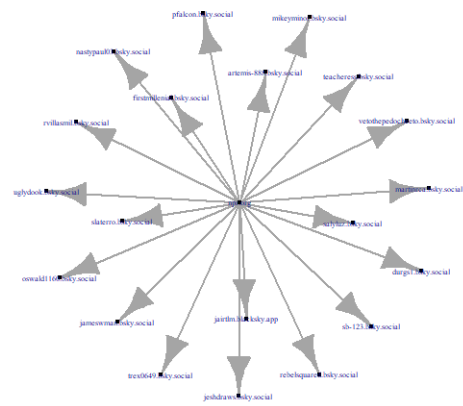
```
library(igraph)
# Graph just the seed user and the people who liked their skeet
g_seed_normal <- graph_from_edgelist(normal_edgelist_seed, directed = T)
g_seed_news <- graph_from_edgelist(news_edgelist_seed, directed = T)
# Graph the full two-level network from the complete edge list
g_normal <- graph_from_edgelist(normal_edgelist,directed=T)
g_news <- graph_from_edgelist(news_edgelist,directed=T)
# Graph the induced subgraph
g2_normal <- induced_subgraph(g_normal,V(g_normal)[degree(g_normal)>1])
g2_news <- induced_subgraph(g_news,V(g_news)[degree(g_news)>1])

# Plot the seed user and direct likes only
par(mfrow=c(1,2))
plot(g_seed_normal, layout=layout_with_fr(g_seed_normal), vertex.size=2, vertex.label.cex=0.25,
  main = "Normal User and Direct Likes")
plot(g_seed_news, layout=layout_with_fr(g_seed_news), vertex.size=2, vertex.label.cex=0.25,
  main = "News User and Direct Likes")
```

Normal User and Direct Likes



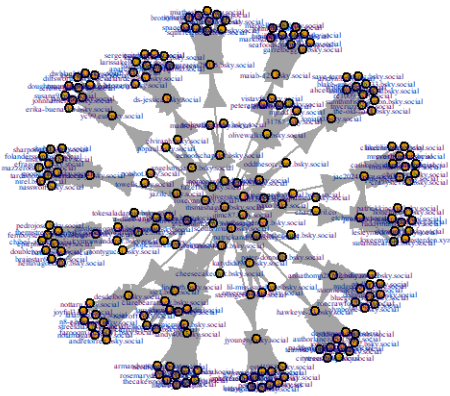
News User and Direct Likes



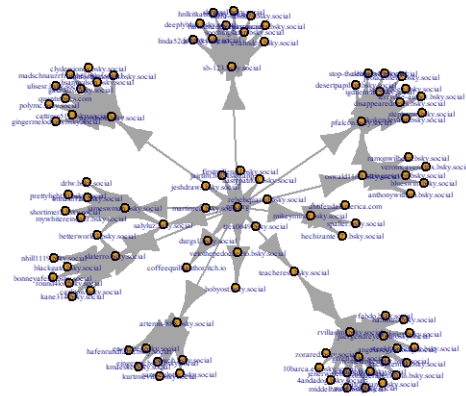
Plot the full network

```
plot(g_normal, layout=layout_with_fr(g_normal), vertex.size=5, vertex.label.cex=0.25,
     main = "Full Normal Likes Network")
plot(g_news, layout=layout_with_fr(g_news), vertex.size=5, vertex.label.cex=0.25,
     main = "Full News Likes Network")
```

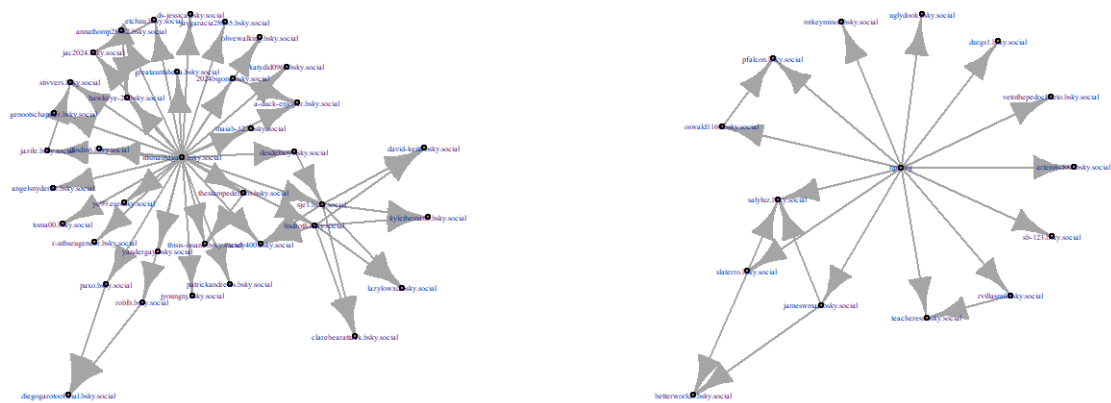
Full Normal Likes Network



Full News Likes Network



```
# Plot the subgraph of shared second-Level Likes with Kamada-Kawai positioning
plot(g2_normal, layout=layout_with_kk(g2_normal), vertex.size=3, vertex.label.cex=0.25,
     main = "Subgraph of Normal Shared Second-Level Likes")
plot(g2_news, layout=layout_with_kk(g2_news), vertex.size=3, vertex.label.cex=0.25,
     main = "Subgraph of News Shared Second-Level Likes")
```



```
## [1] 102
```

```
round(edge_density(g_news), 4)
```

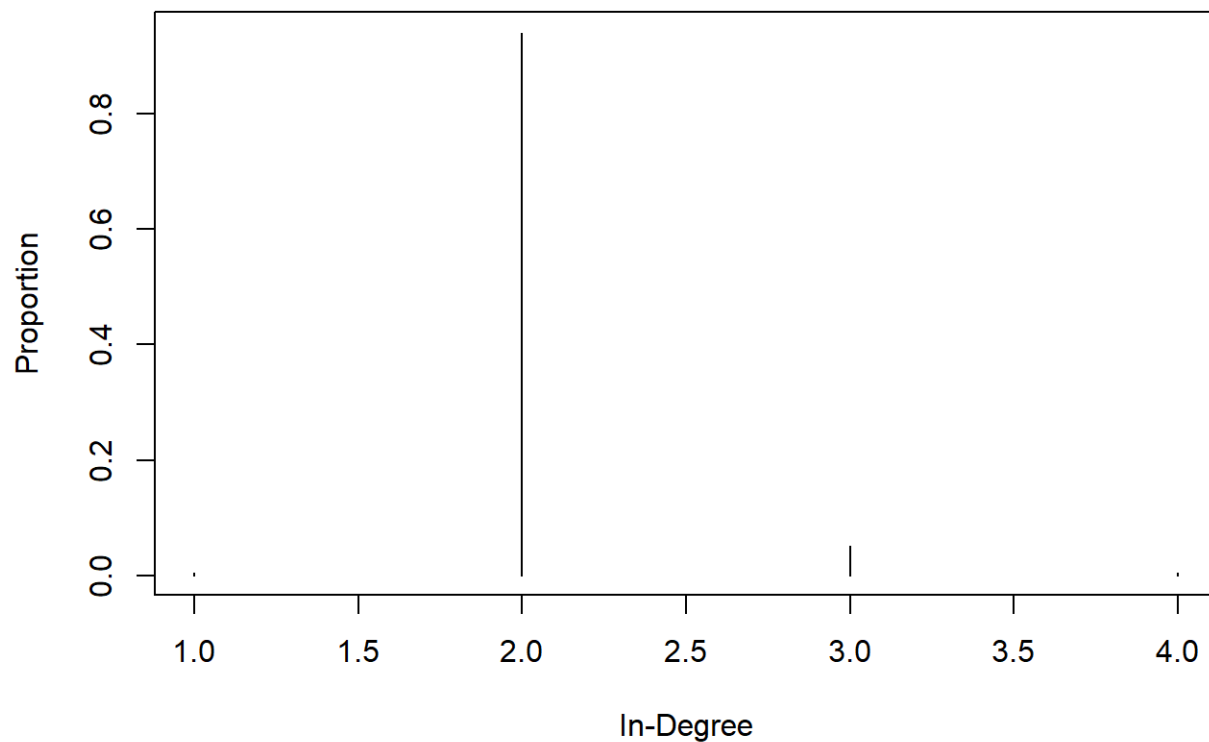
```
## [1] 0.0107
```

The edge density of the normal network is quite small. This tells us that while each node is connected by at least one edge, there is not much or any crossover between the users who have liked posts from different accounts. This is evident in the “clusters” of nodes along the outer edge of the “Normal Full Likes Network”.

The edge density of the news network is also small but almost double that for the normal network. This shows that users who like the news post are more connected than users who liked the normal post.

```
#plot the in-degree distribution for shape comparison
plot(degree_distribution(g_normal, mode = "in"), type="h",
      xlab="In-Degree", ylab = "Proportion",
      main = "Bluesky degree distribution")
```

Bluesky degree distribution



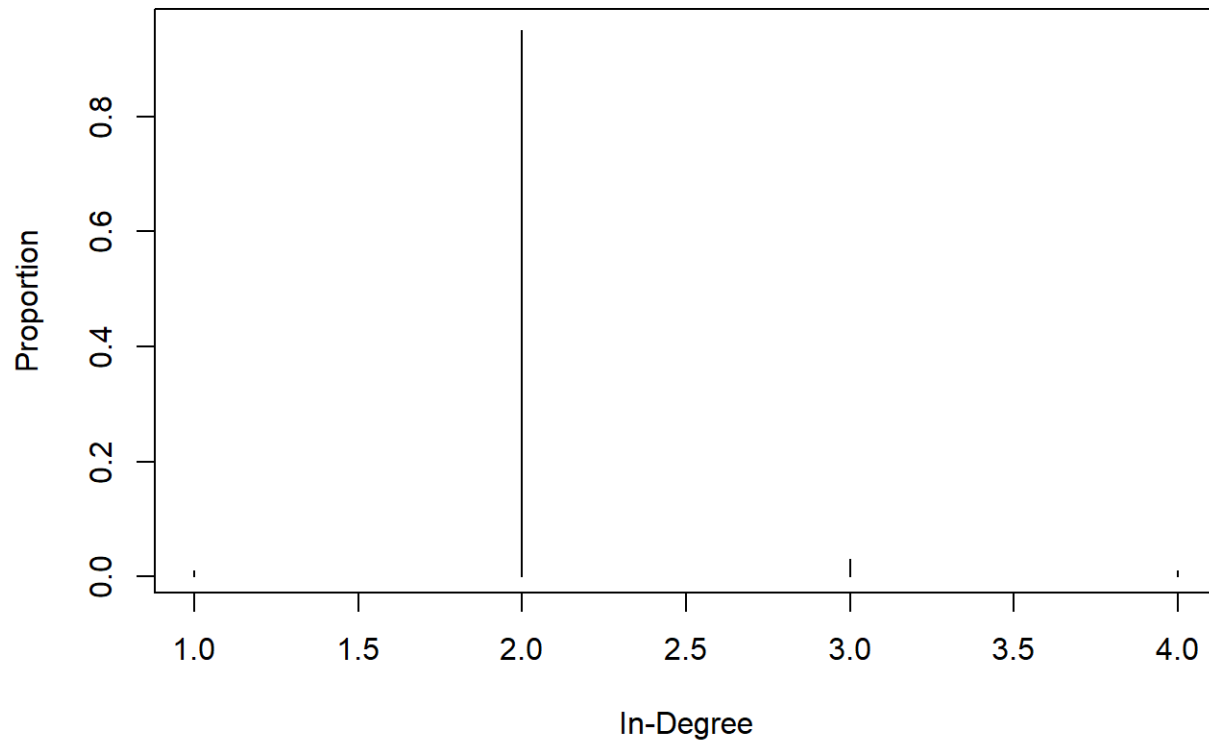
```
print(degree_distribution(g_normal, mode = "in"))
```

```
## [1] 0.004694836 0.938967136 0.051643192 0.004694836
```

This distribution shows us that approximately 90% of the nodes in the normal full network have an in-degree of 2. Even though the edge density of the full graph is low, we could see from the subgraph that users in the second level of likes had also liked each others' posts.

```
#plot the in-degree distribution for shape comparison
plot(degree_distribution(g_news, mode = "in"), type="h",
      xlab="In-Degree", ylab = "Proportion",
      main = "Bluesky degree distribution")
```

Bluesky degree distribution



```
print(degree_distribution(g_news, mode = "in"))
```

```
## [1] 0.01020408 0.94897959 0.03061224 0.01020408
```

Approximately 93% of the users in the news likes network have an in-degree of 2, which is higher than that for the normal network, but less users here have an in-degree of 3.

```
#highest degree scores are most central
head(sort(degree(g_normal), decreasing=T),10)
```

```
## itismashagain.bsky.social      tandy400.bsky.social
##                               39                               13
## hawkeye-23.bsky.social        jac2024.bsky.social
##                               12                               12
## maiab-123.bsky.social javgaracia28035.bsky.social
##                               11                               11
## 2024bigone.bsky.social        desdelboy.bsky.social
##                               11                               11
## ds-jessica.bsky.social        tonia00.bsky.social
##                               11                               11
```

From degree scores, the user `itismashagain.bsky.social` is the most central.

```
#highest igraph closeness scores are most central
round(head(sort(closeness(g_normal,mode="all"),
                decreasing=T), 10), 4)
```

```
## itismashagain.bsky.social      sje1.bsky.social
##                               0.0026              0.0018
## hawkeye-23.bsky.social        jac2024.bsky.social
##                               0.0018              0.0018
## tandy400.bsky.social annathomp28782.bsky.social
##                               0.0018              0.0018
## etchuu.bsky.social          desdelboy.bsky.social
##                               0.0018              0.0018
## lindroth.bsky.social        maiab-123.bsky.social
##                               0.0017              0.0017
```

Once again, `itismashagain.bsky.social` is the most central, this time using the igraph closeness centrality.

```
#highest betweenness scores are most central
round(head(sort(betweenness(g_normal,directed=T),
                decreasing=T), 10), 4)
```

```
## tandy400.bsky.social      sje1.bsky.social
##                               40              23
## hawkeye-23.bsky.social        jac2024.bsky.social
##                               20              20
## maiab-123.bsky.social javgaracia28035.bsky.social
##                               10              10
## ds-jessica.bsky.social        tonia00.bsky.social
##                               10              10
## yc99.eurosky.social          yandergay.bsky.social
##                               10              10
```

Using betweenness centrality, the user `tandy400.bsky.social` is the most central. This user did appear in the previous top tens for centrality, but it is surprising that `itismashagain.bsky.social` does not appear in this top ten despite previously being central.

```
# For news user
head(sort(degree(g_news), decreasing=T),10)
```

```
##          npr.org  rvillasmil.bsky.social  pfalcon.bsky.social
##              20              20              12
##  uglydook.bsky.social  sb-123.bsky.social  slaterro.bsky.social
##              11              11              9
##  jameswman.bsky.social  artemis-888.bsky.social  oswald1160.bsky.social
##              8              8              7
##  mikeyminor.bsky.social
##              4
```

From degree scores, the user `npr.org` is the most central.

```
#highest igraph closeness scores are most central
round(head(sort(closeness(g_news,mode="all"),
                  decreasing=T), 10), 4)
```

```
##          npr.org  rvillasmil.bsky.social  pfalcon.bsky.social
##          0.0057          0.0043          0.0041
##  oswald1160.bsky.social  uglydook.bsky.social  sb-123.bsky.social
##          0.0040          0.0040          0.0040
##  teacheress.bsky.social  slaterro.bsky.social  salyluz.bsky.social
##          0.0040          0.0039          0.0039
##  artemis-888.bsky.social
##          0.0039
```

Once again, `npr.org` is the most central, this time using the igraph closeness centrality.

```
#highest betweenness scores are most central
round(head(sort(betweenness(g_news,directed=T),
                  decreasing=T), 10), 4)
```

```
##          pfalcon.bsky.social  rvillasmil.bsky.social
##              20.0              18.0
##  uglydook.bsky.social  sb-123.bsky.social
##              10.0              10.0
##  artemis-888.bsky.social  slaterro.bsky.social
##              7.0              6.5
##  jameswman.bsky.social  oswald1160.bsky.social
##              5.5              5.0
##  mikeyminor.bsky.social  vetothepedocheeto.bsky.social
##              3.0              1.0
```

This time `pfalcon.bsky.social` is most central, and `npr.org` doesn't make the top ten most central.

To conclude, the likes network for the news user is more dense than that for the normal user, but both networks have low density.

Conclusion

This project analysed 1,030 Bluesky posts about the FIFA World Cup 2026 using four methods. The results showed that most posts focused on one news story rather than general World Cup discussions. Common words included “uefa”, “infantino” and “boycott”.

The hypothesis test found a significant difference in engagement between posts mentioning a host country and other posts. Host-country posts had higher average engagement, but only 14 posts mentioned a host country. Therefore, the result should be interpreted carefully.

The clustering analysis also showed that many posts discussed similar topics, with one large cluster containing most of the posts. The network analysis found that the news account's likes network was denser than the ordinary user's network.

The main limitation was that most posts came from a single day and focused on one news story. Therefore, our findings cannot represent all discussions about the FIFA World Cup 2026. Future research could collect a wider range of posts over a longer period to gain a better understanding of public discussion.